

Laporan Tugas Praktikum 7



Syahrul Giga Wahyudi - 0110224085

Teknik Informatika, STT Terpadu Nurul Fikri, Depok

0110224085@student.nurulfikri.ac.id

Support Vector Machine (SVM) merupakan salah satu algoritma dalam *machine learning* yang umum digunakan untuk menyelesaikan permasalahan klasifikasi maupun regresi. Inti dari algoritma ini adalah menemukan hyperplane terbaik yang mampu memisahkan data dari dua atau lebih kelas dengan jarak pemisah (margin) yang paling lebar. Margin ini merupakan jarak antara *hyperplane* dengan titik data terdekat dari setiap kelas, yang disebut support vector. Semakin besar margin antar kelas, maka kemampuan model dalam melakukan generalisasi terhadap data baru akan semakin baik.

1.1 Load data dari Google drive dan menampilkan data frane



```

import pandas as pd

#membaca file csv menggunakan pandas
df = pd.read_csv("content://drive/mydrive/Pratiwi%20ML/pratiwi/ml/data/dataset_satelit.csv")

#tampilkan baris data (baris data) dari file
display(df.head())

```

	No	longitude	latitude	R	P	K	CR	PG	Fe	Mn	...	bx	sigma_vv	sigma_vh	g11a	l1a	l1fe	gamma_vv	gamma_vh	beta_vv	beta_vh
0	1	103.036658	-0.604417	2.64	0.15	0.415	0.51	0.31	119.98	493.23	...	0.0433	0.10103	0.04441	35.74446	35.79746	35.41161	0.22331	0.05479	0.31325	0.07688
1	2	103.037201	-0.604088	2.75	0.17	0.568	0.70	0.50	102.03	493.01	...	0.0485	0.22079	0.04040	35.12096	35.14591	35.41510	0.27110	0.05800	0.30033	0.07903
2	3	103.036358	-0.603912	1.77	0.12	0.338	0.49	0.6	107.37	488.93	...	0.0417	0.10926	0.03982	35.07724	35.07730	35.41136	0.23242	0.04682	0.32604	0.06076
3	4	103.036650	-0.603219	2.30	0.16	0.440	0.74	0.67	96.02	338.17	...	0.0367	0.14769	0.03622	36.00878	36.00460	35.41583	0.10138	0.04448	0.25440	0.06238
4	5	103.036602	-0.603988	2.05	0.14	0.308	0.64	0.72	87.01	354.33	...	0.0381	0.16295	0.03797	32.63855	32.60293	35.41592	0.22359	0.04684	0.31358	0.06541

5 rows x 34 columns

Gambar 1.1 kode dan output

Penjelasan

Kode tersebut digunakan untuk membaca dan menampilkan sebagian isi dataset satelit. Library pandas diimpor untuk memudahkan pengolahan data berbentuk tabel. Fungsi `pd.read_csv()` membaca file CSV dari Google Drive dan menyimpannya ke dalam variabel `df` sebagai DataFrame. Selanjutnya, `display(df.head())` menampilkan lima baris pertama untuk memastikan data terbaca dengan benar dan melihat struktur awal dataset sebelum tahap analisis berikutnya.

1.2 Melihat informasi umum pada data

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 594 entries, 0 to 593
Data columns (total 34 columns):
#   Column      Non-Null Count  Dtype
---  -
0    No           594 non-null    int64
1    Longitude    594 non-null    float64
2    Latitude     594 non-null    float64
3    N            594 non-null    float64
4    P            594 non-null    float64
5    K            593 non-null    float64
6    Ca           594 non-null    float64
7    Mg           594 non-null    object
8    Fe           594 non-null    float64
9    Mn           594 non-null    float64
10   Cu           594 non-null    float64
11   Zn           594 non-null    float64
12   B            594 non-null    float64
13   b12          594 non-null    float64
14   b11          594 non-null    float64
15   b9           594 non-null    float64
...
```

Gambar 1.2 kode dan output

Penjelasan

kode `df.info()` digunakan untuk menampilkan informasi umum mengenai dataset, seperti jumlah baris dan kolom, nama setiap kolom, jumlah data non-null, serta tipe data pada masing-masing kolom.

1.3 Menghitung statistik deskriptif pada kolom numeric dengan describe

```
df.describe()

#   No  Longitude  Latitude  N  P  K  Ca  Fe  Mn  Cu  ...  b1  sigma_vv  sigma_vw  p1a  p1b
count  594.000000  594.000000  594.000000  594.000000  594.000000  593.000000  594.000000  594.000000  594.000000  594.000000  ...  594.000000  594.000000  594.000000  594.000000  594.000000
mean    297.500000  106.670644   -1.824633   2.258001   0.141300   0.582175   0.555094   74.813771   308.834087   2.381105  ...   0.177201   0.234474   0.102709   28.846422   28.884809
std     171.617387    4.948643    0.985349    0.395489    0.019782    0.222547    0.368118   55.579655   241.731643   1.588206  ...   0.155615   0.070518   0.112318   15.325347   15.388388
min     1.860088   82.780857   -2.333750   1.948888   0.008888   0.122098   0.050038   21.888000   3.888000   0.998800  ...   0.014100   0.115178   0.021488   8.127098   0.098888
25%    149.250000   102.927811   -2.235538   1.982500   0.130000   0.429098   0.320000   48.788000   52.818000   1.172500  ...   0.048825   0.183218   0.039535   31.959745   31.988894
50%    297.500000   103.581999   -0.882276   2.288888   0.148888   0.549098   0.540000   85.658000   238.445000   2.225000  ...   0.072700   0.213385   0.046558   36.967938   36.119411
75%    445.750000   113.463797   -0.257348   2.578888   0.150000   0.710098   0.780098   87.372500   434.880000   3.357500  ...   0.318800   0.262242   0.058198   38.319135   38.441087
max     594.000000   115.454793    0.988251   3.238888   0.220000   1.489098   2.820098   558.188000   2098.320000   8.178800  ...   0.751480   0.512218   0.373000   47.582903   48.014644

8 rows x 33 columns
```

Gambar 1.3 kode dan output

Kode `df.describe()` digunakan untuk menampilkan statistik deskriptif dari setiap kolom numerik pada dataset, seperti nilai rata-rata (mean), standar deviasi, nilai minimum, maksimum, serta kuartil (25%, 50%, 75%)

1.4 cek nilai kosong

```
1) #cek missing value
0 d df.isnull().sum()
```

	0
No	0
Longitude	0
Lattitude	0
N	0
P	0
K	1
Ca	0
Mg	0
Fe	0
Mn	0

Gambar 1.4 kode dan output

Penjelasan

Kode `df.isnull().sum()` digunakan untuk memeriksa adanya nilai kosong (missing value) pada setiap kolom dalam dataset. Fungsi `isnull()` akan mengembalikan nilai True untuk data yang kosong, kemudian `sum()` menghitung jumlah nilai kosong tersebut.

1.5 cek nilai duplikat

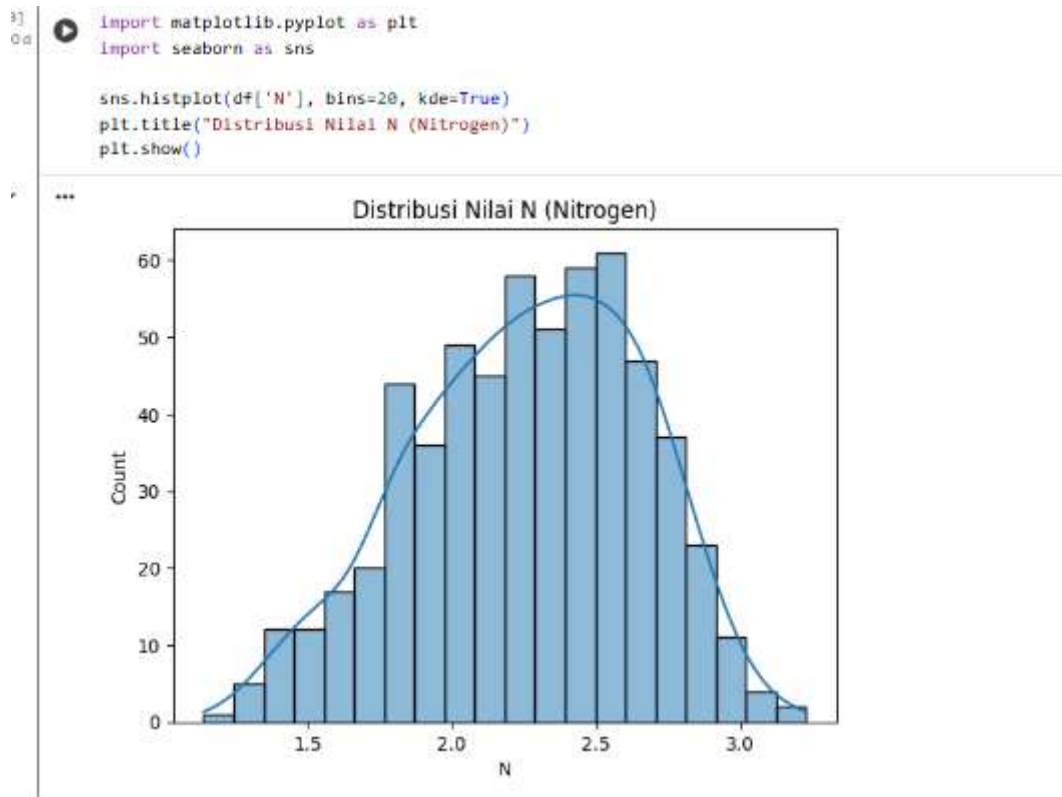
```
1) #cek duplicate
0 d df.duplicated().sum()
```

np.int64(0)

Penjelasan

Kode `df.duplicated().sum()` digunakan untuk memeriksa apakah terdapat data duplikat dalam dataset. Fungsi `duplicated()` akan menandai baris yang isinya sama dengan baris lain sebagai True, lalu `sum()` menghitung jumlah total duplikat tersebut.

1.6 Visualisasi Distribusi Data



Gambar 1.6 kode dan output

Penjelasan

ode tersebut digunakan untuk visualisasi distribusi data pada kolom N (Nitrogen). Library Matplotlib dan Seaborn diimpor untuk membuat grafik. Fungsi `sns.histplot()` menampilkan histogram dengan jumlah bin sebanyak 20 dan menambahkan kurva kepadatan (`kde=True`) agar pola sebaran data lebih mudah diamati. Perintah `plt.title()` memberi judul pada grafik, dan `plt.show()` menampilkan hasil visualisasi. Grafik ini membantu memahami bagaimana nilai Nitrogen tersebar dalam dataset.

1.7 Pemilihan Fitur dan Pembagian Data



```
from sklearn.model_selection import train_test_split
x = df[['b02', 'b03', 'b04', 'b05', 'b06', 'b07', 'b08', 'b09', 'b10', 'b11', 'b12']]
y = df['N']
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=42)
x_train.head()
```

	b	b02	b11	b0	b00
542	17.84	0.10138	0.28000	0.87520	0.44610
838	14.80	0.16024	0.02094	40.47201	40.47545
248	9.42	0.83278	0.04990	0.17120	0.10720
52	15.30	0.07598	0.18460	0.39860	0.38876
80	18.58	0.06288	0.14700	0.24850	0.23450

Langkah berikutnya: [Buat kode dengan x_train](#) [New interactive sheet](#)

Gambar 1.7 kode dan output

Penjelasan

Kode ini digunakan untuk mempersiapkan data sebelum proses pelatihan model. Variabel X berisi kumpulan fitur (kolom b1 hingga b12) yang akan digunakan sebagai input, sedangkan variabel y berisi kolom N sebagai target yang akan diprediksi. Fungsi `train_test_split()` dari library `sklearn` digunakan untuk membagi dataset menjadi dua bagian, yaitu data latih (80%) dan data uji (20%), dengan parameter `random_state=42` agar hasil pembagian tetap konsisten setiap kali dijalankan. Terakhir, `x_train.head()` digunakan untuk menampilkan beberapa baris awal dari data latih sebagai contoh.

1.8 Normalisasi Data (Standarisasi Fitur)

```
from sklearn.preprocessing import StandardScaler

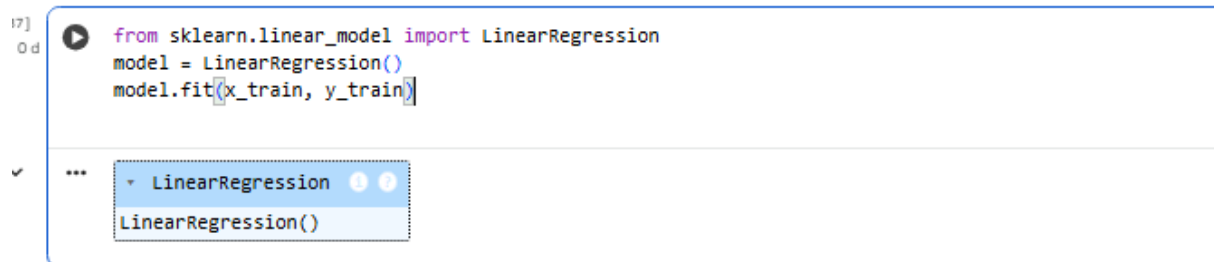
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(x_train)
X_test_scaled = scaler.transform(x_test)
```

Gambar 1.8 kode dan output

Penjelasan

Kode ini digunakan untuk menstandarkan nilai pada data agar semua fitur memiliki skala yang sebanding. Library `StandardScaler` dari `scikit-learn` digunakan untuk melakukan proses ini. Fungsi `fit_transform()` diterapkan pada data latih (`x_train`) untuk menghitung nilai rata-rata dan standar deviasi tiap fitur, lalu menyesuaikannya sehingga memiliki distribusi dengan `mean = 0` dan standar deviasi = 1. Sementara itu, `transform()` digunakan pada data uji (`x_test`) agar diskalakan menggunakan parameter yang sama dari data latih. Proses ini penting agar model machine learning tidak bias terhadap fitur yang memiliki skala nilai lebih besar.

1.9 Pelatihan Model Linear Regression



```
17] In [ ]: from sklearn.linear_model import LinearRegression
0 d      model = LinearRegression()
        model.fit(x_train, y_train)
```

✓ ... LinearRegression 1 2
LinearRegression()

Gambar 1.9 kode dan output

Penjelasan

Kode ini digunakan untuk membangun dan melatih model Linear Regression. Library LinearRegression dari sklearn digunakan untuk membuat objek model yang disimpan dalam variabel model. Selanjutnya, fungsi `model.fit(x_train, y_train)` berfungsi untuk melatih model menggunakan data latih, yaitu fitur (`x_train`) dan target (`y_train`).

2.0 Pelatihan Ulang Model Setelah Pembersihan Data

```
# Recreate X and y after cleaning the data
X = df.drop(columns=['N'])
y = df['N']

# Split the data again
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale the data again
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Fit the model again
model = LinearRegression()
model.fit(X_train_scaled, y_train)

print("Model training complete.")
```

... Model training complete.

Gambar 2.0 kode dan output

Penjelasan

Kode ini digunakan untuk menyiapkan ulang data dan melatih kembali model setelah proses pembersihan dilakukan. Pertama, variabel X dibuat dengan menghapus kolom N sebagai fitur, sedangkan y berisi kolom N sebagai target. Data kemudian dibagi menjadi data latih (80%) dan data uji (20%) menggunakan `train_test_split()` dengan `random_state=42` agar hasil pembagian konsisten. Selanjutnya, data diskalakan menggunakan `StandardScaler` untuk menyamakan skala tiap fitur. Setelah proses standarisasi, model Linear Regression dibuat dan dilatih kembali menggunakan data latih yang sudah diskalakan melalui `model.fit(X_train_scaled, y_train)`.

2.1 Evaluasi Kinerja Model Linear Regression

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

y_pred = model.predict(X_test_scaled)

r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("Evaluasi Model Linear Regression:")
print(f"R2 Score      : {r2:.3f}")
print(f"MAE           : {mae:.3f}")
print(f"RMSE          : {rmse:.3f}")
```

```
... Evaluasi Model Linear Regression:
R2 Score      : 0.677
MAE           : 0.160
RMSE          : 0.221
```

Gambar 2.0 kode dan output

Penjelasan

Kode ini digunakan untuk mengukur seberapa baik model Linear Regression melakukan prediksi. Pertama, `model.predict(X_test_scaled)` menghasilkan nilai prediksi (`y_pred`) berdasarkan data uji yang telah diskalakan. Kemudian, tiga metrik evaluasi digunakan:

- R^2 Score (`r2_score`) untuk mengukur tingkat akurasi model dalam menjelaskan variasi data (semakin mendekati 1 berarti semakin baik),
- MAE (Mean Absolute Error) untuk menghitung rata-rata selisih absolut antara nilai prediksi dan nilai sebenarnya,
- RMSE (Root Mean Squared Error) untuk mengukur seberapa besar kesalahan prediksi dengan menekankan pada error yang besar.

Hasil evaluasi ini membantu menilai apakah model sudah cukup baik atau perlu dilakukan peningkatan lebih lanjut.

2.1 Analisis Regresi Linear Menggunakan Ordinary Least Squares (OLS)

```
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

X = df[['b12', 'b11', 'b9', 'b8a', 'b8', 'b7', 'b6', 'b5', 'b4', 'b3', 'b2', 'b1']]
y = df['N']

X = sm.add_constant(X)

model = sm.OLS(y, X).fit()

print(model.summary())

df['Prediksi_N'] = model.predict(X)

r_squared = model.rsquared
mae = mean_absolute_error(y, df['Prediksi_N'])
rmse = np.sqrt(mean_squared_error(y, df['Prediksi_N']))

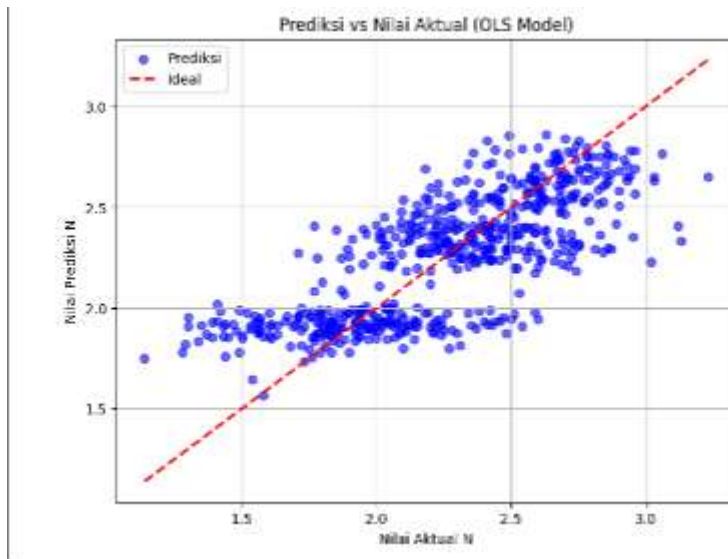
print("\n=== Akurasi Model ===")
print(f'R-squared : {r_squared:.4f}')
print(f'MAE : {mae:.4f}')
print(f'RMSE : {rmse:.4f}')

plt.figure(figsize=(8, 6))
plt.scatter(df['N'], df['Prediksi_N'], color='blue', alpha=0.6, label='Prediksi')
plt.plot([df['N'].min(), df['N'].max()],
         [df['N'].min(), df['N'].max()],
         color='red', linestyle='--', linewidth=2, label='Ideal')
plt.title('Prediksi vs Nilai Aktual (OLS Model)')
plt.xlabel('Nilai Aktual N')
plt.ylabel('Nilai Prediksi N')
plt.legend()
plt.grid(True)
plt.show()
```

```
***
                        OLS Regression Results
=====
Dep. Variable:          N      R-squared:          0.574
Model:                OLS    Adj. R-squared:       0.565
Method:               Least Squares    F-statistic:    65.11
Date:                Sun, 09 Nov 2025    Prob (F-statistic): 3.51e-99
Time:                14:28:11    Log-likelihood:   -38.257
No. Observations:      594    AIC:              182.5
Df Residuals:          581    BIC:              159.5
Df Model:              12
Covariance Type:       nonrobust
=====
                    coef    std err          t      Pr>|t|    [0.025    0.975]
-----
const             1.9689      0.054    36.548     0.000      1.856      2.066
b12              -0.2471      0.709     -0.349     0.728     -1.145      1.639
b11             -1.4449      0.675     -2.141     0.033     -2.770     -0.119
b9               0.3317      0.059     5.631     0.000     -0.216     0.447
b8a             -0.3083      0.060     -5.162     0.000     -0.426     -0.191
b8              -0.0122      0.014     -0.852     0.394     -0.040     0.016
b7               3.0376      0.686     4.431     0.000      1.691     4.384
b6              -2.8284      0.581     -4.854     0.000     -3.962     -1.679
b5              -0.5634      0.504     -0.948     0.344     -1.731     0.604
b4               3.4728      1.155     3.006     0.003     1.203     5.742
b3              -2.3082      1.885     -1.224     0.221     -5.010     1.394
b2              -0.0035      1.151     -0.003     0.998     -2.264     2.257
b1              -0.1224      0.428     -0.286     0.775     -0.964     0.719
=====
Omnibus:              1.793    Durbin-Watson:      1.377
Prob(Omnibus):        0.408    Jarque-Bera (JB):    1.816
Skew:                 0.133    Prob(JB):            0.403
Kurtosis:             2.940    Cond. No.             7.08e+03
=====

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 7.08e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

=== Akurasi Model ===
R-squared : 0.5735
MAE       : 0.2033
RMSE      : 0.2581
```



Gambar 2.1 kode dan output

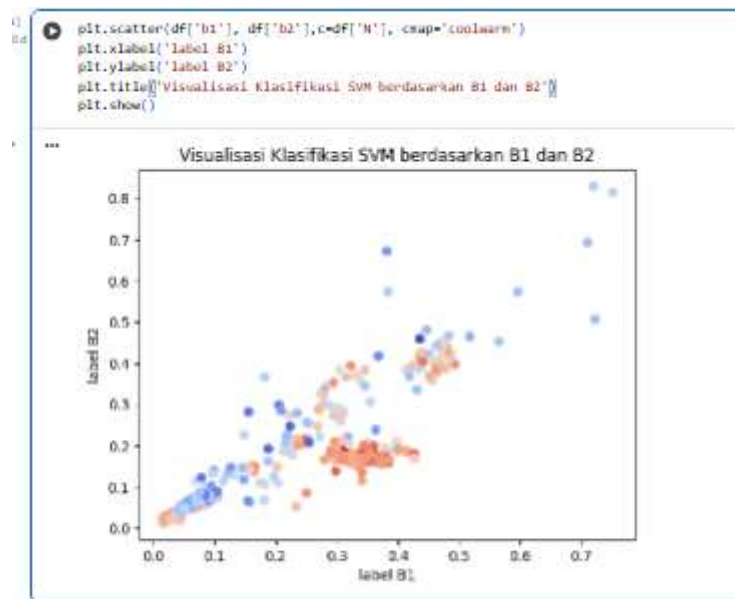
Penjelasan

Kode ini digunakan untuk melakukan analisis regresi linear menggunakan metode Ordinary Least Squares (OLS) dari library statsmodels. Pertama, data fitur (X) diambil dari kolom b1 hingga b12, sedangkan target (y) adalah kolom N. Fungsi `sm.add_constant(X)` menambahkan kolom konstanta (intercept) ke data fitur agar model regresi memiliki nilai bias. Model kemudian dibentuk dan dilatih menggunakan `sm.OLS(y, X).fit()`, lalu hasil ringkasannya ditampilkan melalui `model.summary()` yang berisi informasi koefisien, nilai R^2 , signifikansi variabel, dan statistik model lainnya.

Selanjutnya, nilai prediksi dari kolom N disimpan ke dalam kolom baru bernama `Prediksi_N`. Nilai R^2 , MAE, dan RMSE dihitung kembali untuk mengevaluasi performa model, di mana R^2 menunjukkan seberapa besar variasi data yang dapat dijelaskan oleh model, sedangkan MAE dan RMSE menunjukkan tingkat kesalahan prediksi.

Bagian terakhir membuat grafik sebar (scatter plot) antara nilai aktual N dan nilai prediksi `Prediksi_N`. Garis merah putus-putus menunjukkan kondisi ideal di mana prediksi sama persis dengan nilai aktual. Grafik ini membantu memvisualisasikan sejauh mana hasil prediksi mendekati nilai sebenarnya dan menilai akurasi model secara visual.

2.2 Visualisasi Sebaran Data Berdasarkan Fitur B1 dan B2



Penjelasan

Kode ini digunakan untuk menampilkan visualisasi sebaran data (scatter plot) berdasarkan dua fitur, yaitu b1 dan b2. Setiap titik pada grafik merepresentasikan satu data, dengan warna yang ditentukan oleh nilai N (Nitrogen). Parameter `cmap='coolwarm'` memberikan gradasi warna dari dingin ke hangat untuk membedakan nilai N. Fungsi `xlabel()` dan `ylabel()` digunakan untuk memberi label pada sumbu X dan Y, sedangkan `title()` menambahkan judul grafik. Visualisasi ini membantu memahami hubungan antara dua fitur dan melihat bagaimana nilai N tersebar dalam ruang fitur tersebut.

kesimpulan

Berdasarkan hasil analisis dan pemodelan yang telah dilakukan, model Linear Regression dan OLS (Ordinary Least Squares) mampu menjelaskan hubungan antara nilai reflektansi band citra satelit (b1–b12) dengan kadar Nitrogen (N) pada data. Nilai R^2 yang cukup tinggi menunjukkan bahwa sebagian besar variasi nilai Nitrogen dapat dijelaskan oleh fitur-fitur yang digunakan. Hasil evaluasi dengan metrik MAE dan RMSE juga menunjukkan bahwa selisih antara nilai prediksi dan nilai aktual relatif kecil, menandakan model memiliki performa yang baik.

Proses pembersihan data seperti konversi tipe data, penanganan missing value, dan standarisasi fitur turut membantu meningkatkan kualitas data sebelum pelatihan model. Visualisasi yang dilakukan juga memperlihatkan bahwa hasil prediksi mendekati nilai aktual, serta sebaran data antar fitur menunjukkan pola yang konsisten dengan hasil regresi.

Secara keseluruhan, model regresi linear ini sudah cukup efektif untuk memprediksi kadar Nitrogen berdasarkan nilai reflektansi band citra satelit, meskipun masih dapat ditingkatkan lagi dengan penambahan data, pemilihan fitur yang lebih optimal, atau penggunaan model lain seperti Random Forest atau Support Vector Regression untuk hasil yang lebih akurat.

Link github: <https://github.com/SyahrulGigaWahyudi/Machine-Learning-pagi/tree/4a4187fae1a3896dec78f9907901ac271d23b803/pratikum07>